

SERVERLESS CONTROL PLANE, CONTAINER DATA PLANE: OPERATING AN ENTERPRISE CAUSAL AI PLATFORM

ABSTRACT. Enterprise AI platforms fail in a predictable way: compute scales, tenants multiply, and governance quietly gets smeared across everything that happens to be running. Authorization checks drift into services, tenant scoping leaks into query parameters, and the “platform” becomes a distributed guessing game about identity, tenancy, and intent.

We describe a production architecture that makes the guessing illegal: a *serverless control plane* that materializes tenant context into explicit, cacheable contracts, paired with a *container data plane* that executes workloads without owning policy. Our control plane is an AWS Lambda API (Powertools) fronted by API Gateway. It authenticates users, derives effective context (including a safe **view-as** mechanism), resolves reusable tenant scoping signals (e.g., schema prefixes, tenant identifiers, user/session scope, connector keys, and job intent), minimizes API surface by filtering OpenAPI and endpoint access by enabled modules, and uses ETags and conditional requests to make “who am I?” fast and cheap at scale.

The data plane runs long-lived services on Kubernetes (EKS) and can run burst compute as on-demand containers (e.g., ECS/Fargate). The key property is not the stack; it is the boundary: the control plane decides and attests; the data plane executes. We discuss design goals, contracts, middleware structure, defense-in-depth against confused-deputy and tenant-confusion risks, and operational outcomes that come from keeping governance thin, explicit, and boring.

CONTENTS

1. Introduction	1
2. Problem Statement and Design Goals	2
3. Architecture Overview	4
4. Control Plane Design	6
5. Container Data Plane Integration	10
6. Security and Defense-in-Depth	20
7. Operations and Reliability	22
8. Evaluation	24
9. Related Work	29
10. Limitations and Future Work	29
11. Conclusion	31
References	31

1. INTRODUCTION

A serious platform is a collection of promises: *this tenant’s data stays with this tenant, this user can do this action, this module exists for this role, this request is scoped correctly*. Most systems keep those promises until the product grows. Then governance becomes folklore and enforcement becomes opportunistic: a check in the UI, a check in a service, a check in a gateway, a check in someone’s memory.

We take the opposite approach: instead of letting policy leak into the compute fabric, we concentrate governance into a *thin, serverless control plane* that produces explicit contracts. The

control plane is not “the backend.” It is a very specific thing: an identity- and tenancy-aware decision layer that:

- resolves *who* is calling and *as whom* they are operating (**view-as**),
- derives tenant scoping signals that downstream systems must not guess,
- controls which platform modules and APIs are visible/accessible for the effective context,
- emits cache-friendly responses aligned with HTTP semantics (ETag + **If-None-Match**) [17].

The data plane is everything that runs compute: Kubernetes services, batch jobs, workers, schedulers, and on-demand tasks. It does not decide what a tenant is. It receives a scoped context and does work.

Why this matters. In causal AI and enterprise analytics platforms, the expensive part is not the UI. It is the *combination* of: multi-tenant isolation, module-level capability boundaries, and long-running compute. If tenant scoping is inconsistent, you do not have a platform; you have a roulette wheel with logs.

Contributions. This paper contributes:

- **A practical control-plane pattern:** serverless governance (API Gateway + Lambda) that materializes tenant context and exposes versioned contracts [1, 12, 13].
- **A contract-first API surface:** `/v2/me` as a materialized context contract, plus OpenAPI minimization to keep the visible surface aligned with enabled modules.
- **Defense-in-depth against tenant confusion:** explicit actor vs effective context, allow-listed **view-as**, and server-injected scoping signals (e.g., schema prefixes).
- **Operational lessons:** caching strategy, blast-radius reduction, and incident clarity when the “truth” about tenancy lives at the edge.

A small amount of humor is allowed, because without it, this paper would be a tragedy.

2. PROBLEM STATEMENT AND DESIGN GOALS

2.1. The problem: governance smeared across the runtime. In multi-tenant systems, “tenant” is both a business concept and a security boundary. The main failure mode is not a spectacular vulnerability; it is *inconsistency*: one component thinks the tenant is **acme**, another component thinks it is **demo-acme**, and a third component accepts a user-provided **schema** hint.

This creates three recurring classes of incidents:

- **Policy sprawl:** authorization and capability checks scattered across services, gateways, and UI layers.
- **Confused deputy:** a service acts on behalf of a tenant it did not intend to, because it trusted caller-provided scoping parameters [18].
- **Surface drift:** documentation and client behavior diverge from what is actually enabled/allowed (e.g., stale OpenAPI specs or endpoints that exist but should not be reachable).

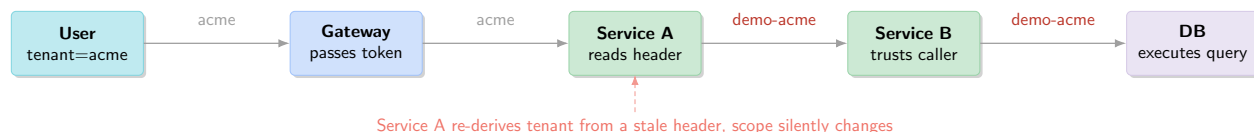


Figure 1. A single request shows the problem: the user intends **acme**, but Service A re-derives tenant from a stale header and forwards **demo-acme**. Service B trusts the caller. The DB query runs against the wrong tenant’s data.

Kubernetes and container orchestrators are excellent at running workloads. They are not optimized to be the single source of truth for tenant identity and policy. When the cluster becomes the

governance layer, every runtime change becomes a policy event. That is how you get “just one more admission policy” until nothing moves.

2.2. Design goals. We adopt five goals that reflect both engineering and operations:

- (1) **G1 — Explicit context:** every request has an immutable *actor* identity and an explicit *effective context*. If `view-as` exists, it is visible and auditable.
- (2) **G2 — Server-injected scoping:** tenant scoping signals (e.g., schema prefixes) are derived by the control plane and injected downstream. Client-supplied scoping parameters are treated as untrusted input.
- (3) **G3 — Minimized surface:** the platform should not advertise APIs and modules that are not enabled for the effective context. OpenAPI and endpoint access are filtered accordingly.
- (4) **G4 — Cache-friendly contracts:** context contracts such as `/v2/me` should support conditional requests (ETag/If-None-Match) to reduce repeated payload transfer [17].
- (5) **G5 — Operational clarity:** logs and traces must encode both actor and effective tenant context so that incidents can be debugged without guessing.

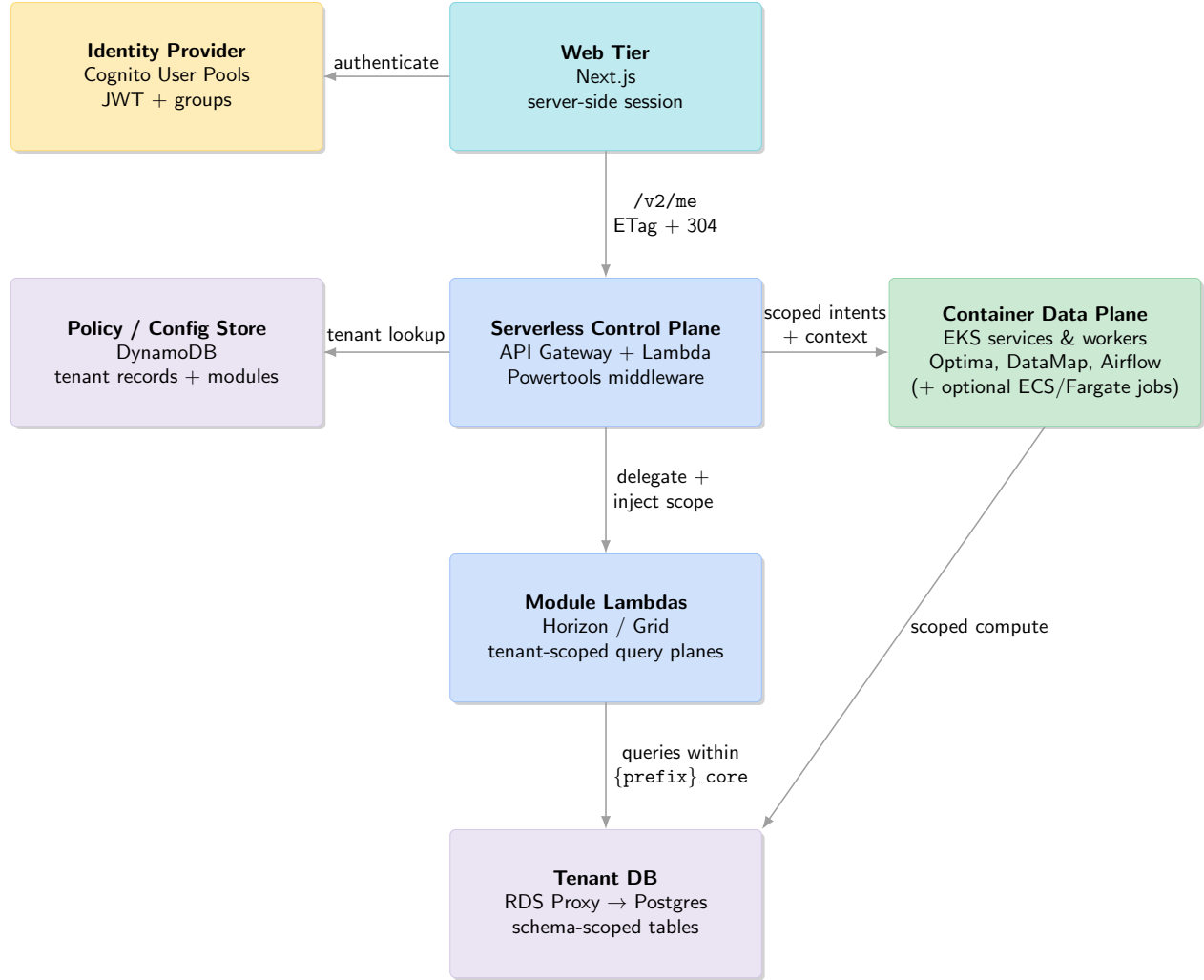
These goals are intentionally conservative. Multi-tenant platforms do not need cleverness. They need consistency. Cleverness is reserved for the causal models.

Capability	Scattered governance	CP/DP split
Tenant isolation	✓	✓
Role-based access	✓	✓
Authentication (JWT)	✓	✓
Single source of tenant context	✗	✓
Server-injected scoping	✗	✓
Cacheable context contracts	✗	✓
Minimized API surface	✗	✓
Actor vs. effective separation	✗	✓
Auditable <code>view-as</code>	✗	✓
Module-gated endpoints	✗	✓
Zero-redeploy tenant config	✗	✓
End-to-end request attribution	✗	✓
Schema-prefix isolation (DB)	✗	✓
Confused-deputy prevention	✗	✓
Substrate-agnostic governance	✗	✓
Filtered OpenAPI per tenant	✗	✓
ETag-based conditional caching	✓	✓
Security headers (CSP/HSTS)	✓	✓
Centralized audit logging	✗	✓
Async heavy compute (data plane)	✓	✓

Table 1. Capability comparison: scattered governance (policy embedded in services) versus the serverless control plane / container data plane split described in this paper. Both provide basic security; the CP/DP split adds consistency, auditability, and operational clarity.

3. ARCHITECTURE OVERVIEW

3.1. Components and boundary. Figure 2 summarizes the system boundary: the control plane decides and attests; the data plane executes.



Boundary rule: tenancy and capability decisions originate in the control plane. Module Lambdas and the container data plane receive explicit scope (e.g. schema prefix) and execute workloads.

Figure 2. Architecture overview. The control plane materializes identity and tenant scope into explicit contracts. Module Lambdas serve interactive queries with injected scope; the container data plane (including Optima) runs long-lived and heavy compute but does not own policy.

3.2. The core contract: `/v2/me`. The control plane’s foundational endpoint is `/v2/me`. It returns: (i) immutable actor identity, (ii) effective context (possibly `view-as`), and (iii) derived scoping signals required by downstream services.

```

1 {
2   "actor": {
3     "email": "ada@acme.example",
4     "tenant_id": "causify",
5     "user_role": "admin",
6     "groups": ["causify:admin", "acme:member"]
7   },
8   "view_as": {
9     "view_as_self": false,
10    "tenant_id": "acme",
11    "user_role": "member",
12    "schema_prefix": "acme"
13  },
14  "impersonatable": [
15    { "tenant_id": "acme", "roles": ["demo", "member", "admin"] }
16  ]
17 }

```

Listing 1. Representative `/v2/me` contract (simplified). Note the explicit actor vs effective context and derived `schema_prefix`.

Caching semantics. This endpoint is called frequently by the web tier. We therefore align it with HTTP semantics: responses include `ETag` and respect `If-None-Match` to return `304 Not Modified` when the effective contract has not changed [17]. This makes “tell me who I am” a cheap question, which is important because the UI asks it like an anxious intern.

Figure 3 illustrates the two paths: a cache miss (full response with a fresh `ETag`) and a cache hit (`304 Not Modified` with no body).

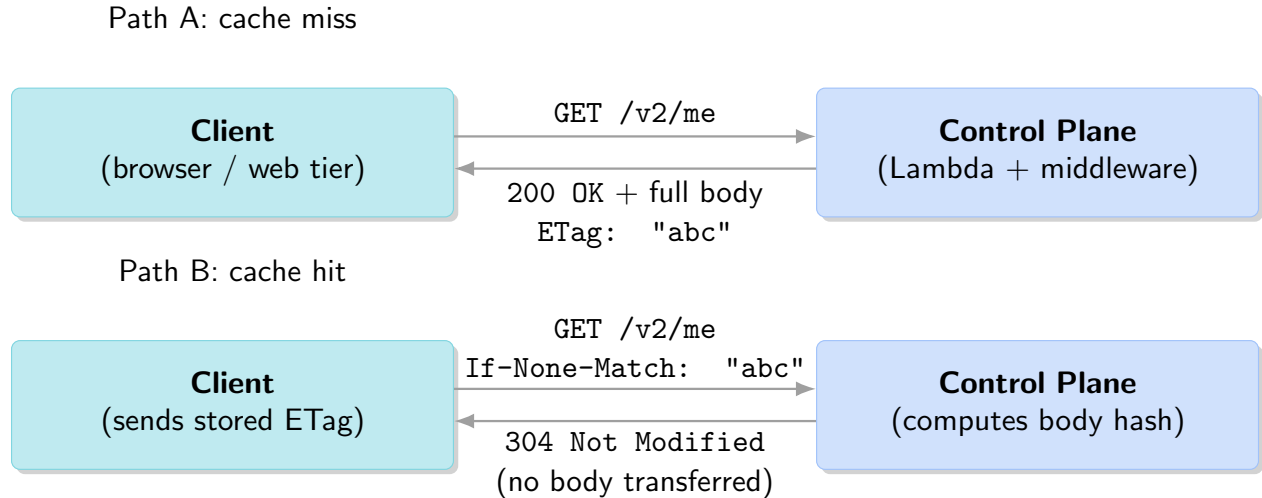


Figure 3. ETag-based caching for `/v2/me`. Path A delivers a full response with an `ETag`; Path B revalidates and returns `304` when the contract is unchanged, saving bandwidth and serialization cost.

3.3. Surface minimization: filtered OpenAPI and module gating. The control plane does not merely *deny* unauthorized endpoints; it also avoids advertising them:

- **Filtered OpenAPI:** `/v1/openapi.json` is generated and then filtered so only endpoints corresponding to enabled modules are exposed in the spec.

- **Module-gated routers:** module endpoints are decorated with a guard (e.g., `@requires_module("horizon")` that checks effective enabled modules.

This reduces surface drift: clients and docs stop describing APIs that should not exist for the effective context.

3.4. Downstream scoping: schema prefix injection. A common confused-deputy risk is tenant scoping provided by the client. We avoid that by deriving `schema_prefix` (and similar scoping signals) in the control plane and injecting them into downstream calls. This ensures that the data plane does not need to infer tenant identity from user-provided parameters.

3.5. Endpoint map (what lives where). Table 2 clarifies responsibilities and helps keep this paper distinct from the UI-as-Policy narrative: here the star is not layout composition; it is governance contracts and scoping.

Endpoint	Purpose	Why it matters
<code>/v2/me</code>	Materialize actor + effective context + scoping signals	Makes tenancy explicit; supports ETag/304 to reduce churn
<code>/v1/openapi.json</code>	Serve minimized OpenAPI surface	Prevents surface drift and accidental client discovery of disabled APIs
Module routers (e.g., <code>/v1/horizon/*</code>)	Serve module APIs with module guards	Enforces capability boundaries consistently per effective context

Table 2. Control plane endpoint responsibilities. This paper focuses on governance contracts and tenant scoping, not UI composition.

4. CONTROL PLANE DESIGN

The control plane is the platform’s *governance cortex*: it authenticates, derives effective tenant context, materializes module availability, and produces contracts that downstream components can execute without guessing. The guiding rule is simple:

Downstream components may execute. They may not improvise tenancy.

4.1. Endpoint taxonomy: contracts, routers, and delegation. The control plane exposes three categories of endpoints:

- **Context contracts (identity & scoping):** `/v2/me` returns immutable actor identity, effective context (including `view-as`), and derived scoping signals such as `schema_prefix`.
- **Surface description:** `/v1/openapi.json` returns an OpenAPI document filtered so clients only see endpoints corresponding to enabled modules.
- **Module routers:** `/v1/horizon/*`, `/v1/grid/*`, and `/v1/datamap/*` implement module APIs guarded by module availability checks.

Critically, some module routers in the control plane act as *delegators*: they forward requests to module-specific Lambda backends (e.g., Horizon and Grid) while injecting derived scoping values. This makes the control plane the only place where tenant scope is *decided*, even when the work is done elsewhere.

4.2. Middleware spine: three decorators that do 80% of the job. The control plane uses Powertools’ REST resolver with a small but decisive middleware chain [13]. The order matters:

```
1 @auth_middleware
2 @etag_middleware
3 @response_headers_middleware
4 @logger.inject_lambda_context(...)
5 def lambda_handler(event, context):
6     return app.resolve(event, context)
```

Listing 2. Middleware chain used by the control plane Lambda. Auth runs first; response shaping and conditional requests run around the handler.

Auth middleware (truth injection). `auth_middleware` validates JWTs, derives `tenant_id` and role from Cognito groups, computes an explicit allow-list of impersonation targets, and parses/validates the semicolon-delimited `view-as` header:

`X-View-As: tenant_id=<id>;role=<role>[;mode=<...>]`

If `view-as` is present, it is validated against `available_impersonations` before it is accepted. The middleware then resolves the tenant record and injects `schema_prefix` into the request claims.

This is a deliberate design choice: downstream components should never accept `schema_prefix` (or equivalent scoping) from the client. That is the confused-deputy trap.

Identity derivation: the `tenant_id:role` convention. Cognito user-pool groups follow a strict naming convention: `tenant_id:role` (e.g., `causify:admin`, `acme:member`). The auth middleware splits each group on `:`, filters for the requested tenant, extracts the role portion, and selects the highest-precedence role when a user belongs to multiple groups for the same tenant.

```
1 valid_groups = [g for g in user_groups if ":" in g
2                 and g.startswith(f"{tenant_id}:")]
3 user_roles = [g.split(":")[1] for g in valid_groups]
4 order = ["admin", "member", "demo"]
5 user_roles.sort(key=lambda r: order.index(r) if r in order else len(order))
6 user_role = user_roles[0]
```

Listing 3. Role derivation from Cognito groups: groups follow a `tenant_id:role` convention and are resolved by precedence.

This convention is load-bearing: it lets the control plane derive both tenant identity and role from a single JWT claim without an external policy lookup, and it makes the impersonation allow-list computable from the same group list.

Response headers middleware (hardening + ETag generation). `response_headers_middleware` adds baseline security headers (`X-Frame-Options: DENY`, strict CSP, HSTS, etc.) and sets a short caching directive (`Cache-Control: public, max-age=300`). It also computes an ETag as an MD5 hash over the response body. (The ETag is quoted per HTTP conventions.)

ETag middleware (cheap revalidation). `etag_middleware` implements conditional requests: when `If-None-Match` matches the current response body hash, the handler returns `304 Not Modified` with the same standard headers. This makes the platform’s most frequently asked question (“who am I, and what tenant am I in?”) cheap [17].

If the UI asks this endpoint a lot, that’s not a bug. That’s anxiety. We treat it like a first-class workload.

Request lifecycle. Figure 4 makes the middleware execution order concrete: a request passes through authentication, conditional caching, and response hardening before reaching module routing.

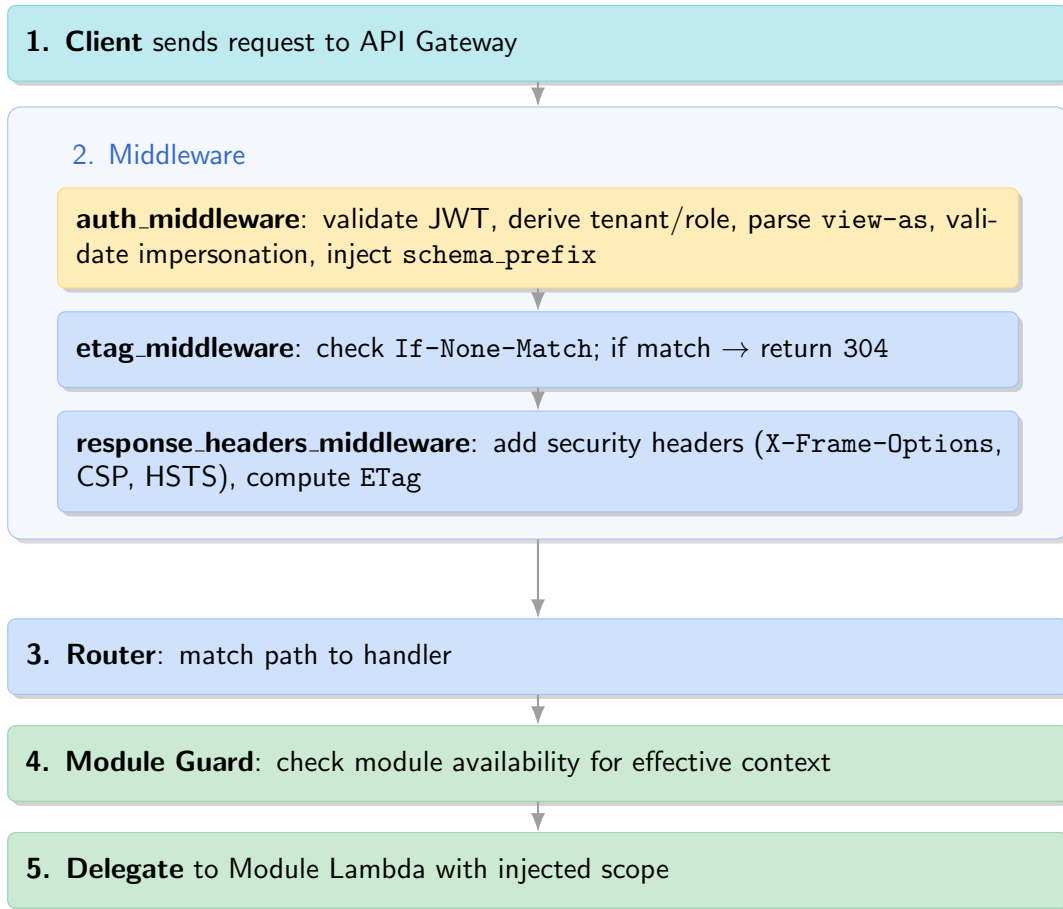


Figure 4. Request lifecycle through the control plane middleware spine. Auth runs first (truth injection), followed by conditional caching, response hardening, routing, module gating, and delegation.

4.3. Module availability as a first-class primitive. Module availability is computed from tenant configuration stored in DynamoDB [2]. The function `get_available_modules(tenant_id, user_role)` returns:

- **modules:** enabled module identifiers (e.g., `horizon`, `grid`, `datamap`)
- **submodules:** enabled sub-module identifiers and their parent mapping (e.g., `horizon: supply-chain`)
- **module_layout:** module-specific JSON fragments stored as JSON strings in DynamoDB and parsed server-side (strict parsing; invalid JSON fails fast).

This strict parsing matters operationally: configuration that silently half-loads is worse than configuration that fails loudly. Loud failures are debuggable.

4.4. Surface minimization: filtered OpenAPI, not just forbidden endpoints. Most APIs only deny. The control plane also *stops advertising*. The OpenAPI endpoint generates the full schema and then filters it:

- Always allow `user` and `tenant` tags.
- Allow module endpoints only when the endpoint tags intersect enabled modules.
- Remove unused component schemas after filtering.

```

1 default_allowed_tags = ["user", "tenant"]
2 is_default_allowed = any(tag in default_allowed_tags for tag in operation_tags)
3 is_module_allowed = any(tag in available_modules for tag in operation_tags)
4 if is_default_allowed or is_module_allowed:
5     keep(path, method)

```

Listing 4. OpenAPI filtering logic: paths are kept if their tags are default-allowed or enabled by module availability.

This reduces surface drift: the platform’s published interface becomes a function of tenant capability, not a static document with wishful thinking.

4.5. Module guards: authorization at router boundaries. Module routers are protected by a `@requires_module(module)` decorator. The decorator evaluates module availability for the *effective context* (actor or view-as) and raises a forbidden error when a module is not enabled.

This is intentionally blunt: it is not trying to infer fine-grained permissions. It ensures the coarse-grained capability boundary is enforced everywhere, including at delegating routers.

While `@requires_module` checks the parent module, a tenant may have that module enabled while only some *submodules* are enabled: core product surfaces, optional add-ons, or sub-features rolled out in stages. For a finer gate, routes use `@requires_submodule("module/submodule")` (for example `@requires_submodule("horizon/cpg")` for CPG submodule APIs under Horizon). The decorator evaluates the effective context against `get_available_modules(tenant_id, user_role)`, which returns tenant `submodules` from DynamoDB together with `modules`; the submodule must be allowed even when the parent module is already on.

```

1 @requires_module("horizon")
2 def list_kpis(...):
3     return invoke_horizon_lambda("/kpis", query_params)
4
5 @requires_module("horizon")
6 @requires_submodule("horizon/cpg")
7 def list_cpg_connectors(...):
8     return invoke_horizon_lambda("/cpg/connectors", query_params)

```

Listing 5. Module guard enforces capability boundaries using the effective tenant/role context. Submodule routes add `@requires_submodule`.

4.6. Delegation with scope injection: control plane as a tenancy firewall. When module routers delegate to module-specific backends (Horizon/Grid Lambdas), the control plane injects `schema_prefix` into query parameters:

```

1 claims = get_current_event().get("claims")
2 schema_prefix = claims.get("schema_prefix")
3 query_params["schema_prefix"] = schema_prefix
4 response = invoke_lambda(horizon_lambda_function, path, query_params, method, body)

```

Listing 6. Delegation pattern: the control plane injects derived tenant scope into downstream module backends.

This protects against the most common tenant-isolation footgun: a downstream service that trusts client-provided scoping parameters. Here, the client never gets the chance.

Mechanics: constructing an API Gateway event. The delegation is not an HTTP call. The control plane constructs an API Gateway-compatible event and invokes the downstream Lambda synchronously via `boto3`:

```
1 lambda_event = {
2     "httpMethod": method,
3     "path": path,
4     "queryStringParameters": query_params, # schema_prefix is here
5     "headers": {"Content-Type": "application/json"},
6     "body": json.dumps(body) if body else None,
7 }
8 response = boto3.client("lambda").invoke(
9     FunctionName=function_name,
10    InvocationType="RequestResponse",
11    Payload=json.dumps(lambda_event),
12 )
```

Listing 7. Lambda-to-Lambda delegation: the control plane builds an API Gateway-like event with injected scope and invokes the module backend synchronously.

This design means that module backends never need their own API Gateway deployment: they receive the same event shape they would get from a public gateway, but the caller is always the control plane, and the scope is always server-derived. It also keeps the delegation path testable: any event that matches the shape can be used in integration tests without an actual gateway.

5. CONTAINER DATA PLANE INTEGRATION

If the control plane is the cortex, the data plane is the muscle: it runs services, workflows, and compute-heavy tasks. In CAUSIFY PLATFORM, the data plane is intentionally heterogeneous, because the workloads are heterogeneous.

5.1. Kubernetes services on EKS: long-running platform components. Long-running components live on Kubernetes (EKS) [4]. This includes web-facing services and platform internals.

A representative example is the Causify Console deployment: it runs behind an internal AWS ALB ingress (TLS enforced, WAF attached, Shield enabled, and access logs streamed to S3). The same patterns appear for Airflow’s webserver ingress, which is also internal and protected. Airflow on Kubernetes (workflow plane). Airflow is deployed in the `airflow` namespace with separate components (e.g., scheduler, webserver, dag-processor) and a `git-sync` deployment to pull DAGs. The scheduler is autoscaled via HPA (CPU and memory utilization targets), and operational hygiene is handled via cleanup cronjobs.

Secrets are integrated using an ESO service account annotated for IRSA (`eks.amazonaws.com/role-arn`) to fetch secrets from AWS Secrets Manager. This allows Kubernetes workloads to consume secrets without static credentials.

In short: Kubernetes runs the things that must keep running.

5.2. Module backends as Lambda services: Horizon and Grid. Some module backends are implemented as dedicated Lambda functions (Horizon/Grid) that implement their own routing tables and connect directly to a Postgres database (using Secrets Manager and a proxy endpoint via environment variables).

These backends are deliberately strict about tenant scoping. For example, the Horizon Lambda extracts `schema_prefix` from query parameters and refuses to operate without it (treating it as required input). It then derives schema names such as `{schema_prefix}_core` internally.

This makes an architectural point:

The control plane decides scope; the module backend enforces it locally.

The control plane ensures `schema_prefix` is derived from authenticated context and injected downstream; the module backend ensures it does not run without scope.

5.3. Airflow → ECS/Fargate: burst compute without cluster debt. Not all compute belongs in Kubernetes. For bounded, bursty jobs (pipelines, model runs, data downloads), Airflow dispatches ECS tasks (Fargate by default) [3, 11].

Airflow DAGs for Horizon and Grid use a helper that produces a configured ECS operator. The helper sets CPU/memory, ephemeral storage, and VPC networking (private subnets and security groups by region). It also wraps the operator with retry logic for known ECS capacity failures.

```
1 class EcsRunTaskOperator_withRetry(EcsRunTaskOperator):
2     _KNOWN_ERRORS = ["Capacity is unavailable at this time."]
3     _MAX_NUM_ATTEMPTS = 2
4     _RETRY_DELAY_SECS = 15
```

Listing 8. Airflow launches ECS tasks via a custom operator that retries known capacity/provisioning failures.

```
1 task = EcsRunTaskOperator_withRetry(
2     task_definition=task_definition,
3     launch_type="FARGATE",
4     overrides={
5         "cpu": str(cpu_units),
6         "memory": str(memory_mb),
7         "ephemeralStorage": {"sizeInGiB": ephemeralStorageGb},
8         "containerOverrides": [{"name": task_definition, "command": task_cmd}],
9     },
10    network_configuration={"awsVpcConfiguration": {...}},
11 )
```

Listing 9. The ECS task operator is configured with VPC networking, resources, and long waiter timeouts (pragmatic fix for real-world behavior).

This pattern yields a clean separation: Airflow orchestrates workflows; ECS/Fargate executes job containers; Kubernetes remains focused on long-lived services.

5.4. Why this mixed data plane is intentional. A single execution substrate is aesthetically pleasing and operationally tempting. It is also how you end up running batch workloads in the same cluster as your interactive services and wondering why everything is on fire.

The platform embraces the reality that:

- Kubernetes is excellent for long-running services and controllable scaling.
- Fargate is excellent for isolated burst compute and operational simplicity.
- Lambda is excellent for narrowly scoped APIs and fast fan-out.

The art is not choosing one. The art is choosing boundaries.

5.5. Domain Module Lambdas: Horizon and Grid as Tenant-Scoped Query Planes. The platform uses dedicated module backends for Horizon and Grid as AWS Lambda services. This is not “Lambda because serverless is trendy.” It is Lambda because these modules behave like *query planes*: they must respond quickly, enforce scoping strictly, and scale to spiky interactive traffic without dragging an entire Kubernetes service along for every request.

Horizon Lambda. Horizon is opinionated and explicit: every request must name its tenant scope via `schema_prefix`. If it is missing, the request is rejected early. The Lambda then derives its working schema (`core_schema = {schema_prefix}_core`), and constructs a service layer that executes queries against tenant-scoped tables. To keep the system calm under transient failures, Horizon uses short retry backoff and structured JSON logs to keep debugging deterministic instead of mystical.

```

1 q = self.event.get("queryStringParameters") or {}
2 self.schema_prefix = q.get("schema_prefix")
3 if not self.schema_prefix:
4     raise BadRequestError("schema_prefix is required")
5 self.core_schema = f"{self.schema_prefix}_core"

```

Listing 10. Horizon Lambda enforces tenant scope explicitly: `schema_prefix` is required and used to derive `core_schema`.

Database access: Secrets Manager + RDS Proxy. Operationally, each module Lambda follows a three-step pattern to reach tenant data: (i) fetch database credentials from AWS Secrets Manager [14] at cold start, (ii) connect through an RDS Proxy endpoint over TLS, and (iii) execute parameterized SQL using a small, testable service layer.

```

1 _SECRET_ARN = os.environ["DB_SECRET_ARN"]
2 _PROXY      = os.environ["PROXY_ENDPOINT"]
3 _DB_NAME    = os.environ["DB_NAME"]
4
5 @retryable
6 def build() -> PgClient:
7     sec = boto3.client("secretsmanager")
8         .get_secret_value(SecretId=_SECRET_ARN)["SecretString"]
9     creds = json.loads(sec)
10    conn = psycopg2.connect(
11        host=_PROXY, dbname=_DB_NAME,
12        user=creds["username"], password=creds["password"],
13        sslmode="require",
14    )
15    return PgClient(conn)

```

Listing 11. Database connection via Secrets Manager and RDS Proxy: credentials are never embedded, and the proxy manages connection pooling across Lambda invocations.

This completes the scope propagation story end-to-end: identity is validated in the control plane, a reusable *scope envelope* is attached to downstream execution (i.e., `schema_prefix` is derived and injected into the delegation event), the module Lambda uses that prefix to derive `{schema_prefix}_core`, and the database connection itself goes through a managed proxy that handles connection pooling and IAM-based access. At no point does a client parameter touch the database path.

Grid Lambda. Grid is similarly scope-driven, but its shape is slightly different: it serves time-series analytics, predictions, and derived signals, and uses the schema prefix to select the correct core tables. The design is intentionally boring: a mapping of features to tables and columns is built around the tenant’s core schema, which avoids hardcoding tenant-specific logic into application code.

```

1 core_table = f"{schema_prefix}_core.load_predictions"
2 prob_table = f"{schema_prefix}_core.threshold_cross_probability"

```

Listing 12. Grid Lambda uses `schema_prefix` to select tenant-scoped core tables.

Why Lambdas for these modules? Because these modules sit at a boundary where traffic is bursty and latency matters: interactive dashboards behave like a strobe light, not a steady lamp. Lambda matches that rhythm. Kubernetes excels at steady-state services and background work; module Lambdas excel at sharp edges where cold requests show up at inconvenient times.

The contract with the control plane. Neither Horizon nor Grid is allowed to “discover” its tenant. Both consume scope. In practice, `schema_prefix` is injected by the control plane and forwarded through delegating routers. This is the anti-confused-deputy rule in executable form: *tenancy is derived from identity, not from client parameters*.

5.6. Reusable Scope is more than prefixes: the *scope envelope*. The platform carries a small *scope envelope* across layers so that execution substrates can remain policy-agnostic. Different modules consume different parts of the envelope:

Scope key	Used by	Purpose
<code>tenant_id</code>	Control plane, connectors, DynamoDB models	Canonical tenant identity for governance, job routing, and data isolation
<code>user_id</code>	DataMap sessions/turns, preferences	Per-user session isolation; multi-session without table sprawl
<code>role / groups</code>	Control plane module gates	Capability surface: what can be called/seen for effective context
DB scope (e.g., prefix/core schema <i>and/or</i> tenant_id+RLS)	Horizon/Grid query planes	Maps tenant identity to database partitioning strategy (schema/core tables, and optionally row-level policies)
<code>subject_id</code> , <code>analysis_id</code> , <code>session_id</code>	DataMap	Stable artifact hierarchy: Subject → Analysis → Graph → Versions
<code>connector_key</code> , <code>job_type</code> , <code>job_id</code>	Connectors/DataPull	Job orchestration, reconciliation windows, and per-connector status tracking

Table 3. The *scope envelope* is intentionally small: enough keys for correct execution, not a dump of “everything we know.”

5.7. DataMap Studio: Subject → Analysis → Graph, With Memory. DataMap is the platform’s causal exploration workbench: a guided flow that turns datasets into structured hypotheses and causality graphs without forcing users to juggle a zoo of unrelated artifacts.

The studio is organized around a simple hierarchy:

Subject → Analysis → Graph (and each Graph can have **Versions**)

This hierarchy is not aesthetic; it is a data model that prevents sprawl. It supports: (i) multiple analyses per subject, (ii) multiple graph versions per analysis, and (iii) per-user, multi-session workflows without multiplying tables per feature.

Refine-in-place, but keep history. DataMap behaves like a whiteboard that remembers. Users can refine nodes/edges in place, then view version history and roll back if an iteration gets weird. This makes experimentation safe: curiosity is encouraged, and regret is supported.

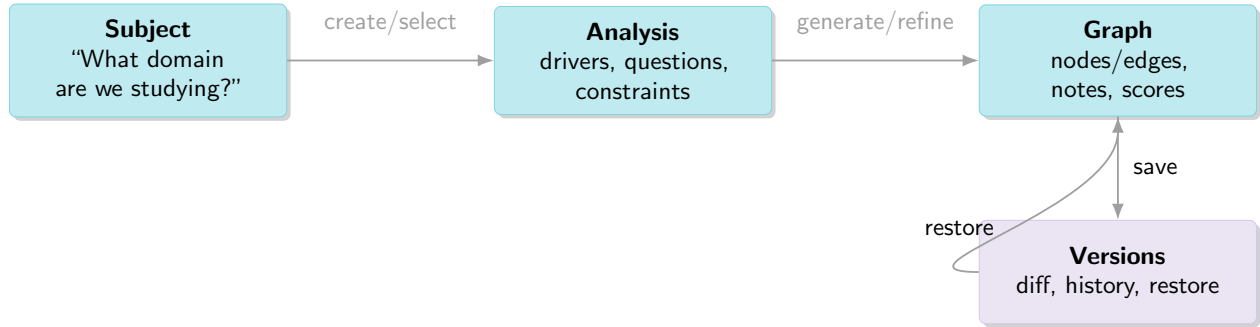


Figure 5. DataMap Studio flow: a stable artifact hierarchy that supports iteration without data-model sprawl.

Any data source. DataMap treats ingestion as a pluggable boundary. A dataset upload is just one implementation of “data source.” The same workflow can support connectors and query-backed sources without changing the studio’s conceptual model.

Multi-session without table sprawl. Instead of creating new tables for every “session” or feature variant, DataMap stores sessions and versions as *keyed artifacts* attached to the Subject/Analysis hierarchy. In practice, this supports:

- multiple concurrent sessions per user,
- multiple users working in parallel,
- and a clean audit trail of graph evolution.

The result is a studio that feels fluid while staying structurally disciplined. It is creativity with guardrails.

One table, several item types. DataMap uses a single DynamoDB table per environment (e.g., `causify_console_datamap_{env}`) that stores a small set of item types: **SUBJECT**, **ANALYSIS**, **GRAPH** (versioned), **SESSION**, and **TURN**. The partition/sort keys are compositional, so the hierarchy is queryable without joins.

```

1  SUBJECT
2  PK: TENANT#<tenant_id>#SUBJECT#<subject_id>
3  SK: SUBJECT
4  Attrs: subject_type, subject_key (JSON), display_name, description,
5         s3_uri_metadata, status, created_ts, created_by, ...
6
7  ANALYSIS
8  PK: TENANT#<tenant_id>#SUBJECT#<subject_id>
9  SK: ANALYSIS#<analysis_id>      (analysis_id = ULID/UUID)
10 Attrs: title, created_ts, created_by_user, last_graph_id, tags[]
11
12 GRAPH (versioned output)
13 PK: TENANT#<tenant_id>#SUBJECT#<subject_id>#ANALYSIS#<analysis_id>
14 SK: GRAPH#<version_ts_desc>      (store as ~ts or negative epoch for newest-first)
15 Attrs: graph_id (ULID), graph_json OR s3_key_graph_json,
16        summary[], changes[] (refine deltas),
17        model, prompt_version, temperature, trace_id,
18        created_by_user, session_id, size_bytes, optional ttl
19
20 SESSION (per-user working session)
21 PK: TENANT#<tenant_id>#USER#<user_id>
22 SK: SESSION#<session_id>

```

```

23  Attrs: subject_id, analysis_id (optional), started_ts, last_graph_id,
24        status (active|closed), ttl
25
26  TURN (multi-turn chat/history)
27  PK: TENANT#<tenant_id>#SESSION#<session_id>
28  SK: TURN#<seq> (001, 002, ... or timestamp)
29  Attrs: role (user|system|assistant|...), content, result_graph_id (optional),
30        trace_id, optional ttl
31

```

Listing 13. DataMap key patterns (verbatim structure).

The key design is compositional: partition keys encode tenancy and hierarchy; sort keys encode artifact identity and time.

Versioning without drama. Graph versions store either embedded graph JSON or a pointer to an S3 key. They also store refine deltas (`changes[]`), summaries, and trace metadata (`trace_id`, `prompt_version`, `temperature`) so the system can explain not just *what* changed, but *why*.

Indexes that match the product. The GSIs follow the user journeys:

- **Recent sessions per user:** GSI1PK = TENANT#<tenant_id>#USER#<user_id>, GSI1SK = TS#<started_ts>.
- **Latest graph per analysis:** GSI2PK = TENANT#<tenant_id>#SUBJECT#<subject_id>#ANALYSIS#<analysis_id>, GSI2SK = GRAPH#<version_ts_desc> (query with limit=1).
- **All graphs by user:** GSI3PK = USER#<user_id>, GSI3SK = TS#<version_ts_desc>.

Storage symmetry with S3. Datasets are stored read-only in S3; exports are organized by tenant/-subject/analysis/graph prefixes, with server-side encryption and blocked public access. DynamoDB stores the “shape” and history; S3 stores the heavy artifacts. TTL can be used for demo cleanup where appropriate.

This model yields to our desired studio behavior: refine in place, view history, restore earlier versions, support multiple analyses per subject, and allow many concurrent user sessions—all without creating new tables every time the product team has an idea.

The studio remains fast because the “truth” is indexable, and the “bulk” is object storage.

5.8. Tenant meta as data: membership, connectors, billing, preferences, audit. Beyond modules and compute, a multi-tenant platform needs “tenant meta” primitives: who belongs to a tenant, which connectors are configured, whether billing allows a module, and what defaults a user prefers. We model this in DynamoDB with small, purpose-built tables and GSIs that match access patterns:

- **Membership/seats:** PK=tenant_id, SK=user_id with GSI by email to resolve invites and multi-tenant users.
- **Invites:** unguessable invite_token as primary key; on acceptance, create membership row and add Cognito group {tenant_id}:{role}.
- **Connector state:** PK=tenant_id, SK=connector_key storing status, last sync timestamps, and a reference to credentials stored in Secrets Manager (not raw secrets).
- **Billing:** one row per tenant with GSIs for Payment Gateway (e.g., Stripe) customer/subscription lookup so webhooks can map events → tenant deterministically.
- **Preferences:** PK=tenant_id, SK=user_id for default modules, locale/theme, etc.
- **Audit log:** PK=tenant_id, SK=timestamp#uuid with action_type and payload.

The platform benefit is operational: these models make onboarding, licensing, connector operations, and UX defaults *data operations* rather than application forks.

5.9. Connector ingestion case study: control plane + job intent + Kubernetes workers.

Connectors are where multi-tenancy fails quietly unless scope is explicit. A reusable pattern emerges in connector-heavy multi-tenant products (e.g., CPG connectors such as Shopify/Amazon): *keep the connector hot path API-driven and multi-tenant by design*, and use workflow engines only for offline analytics. Our ingestion architecture therefore mirrors the paper’s thesis: *thin control plane, heavy execution plane*.

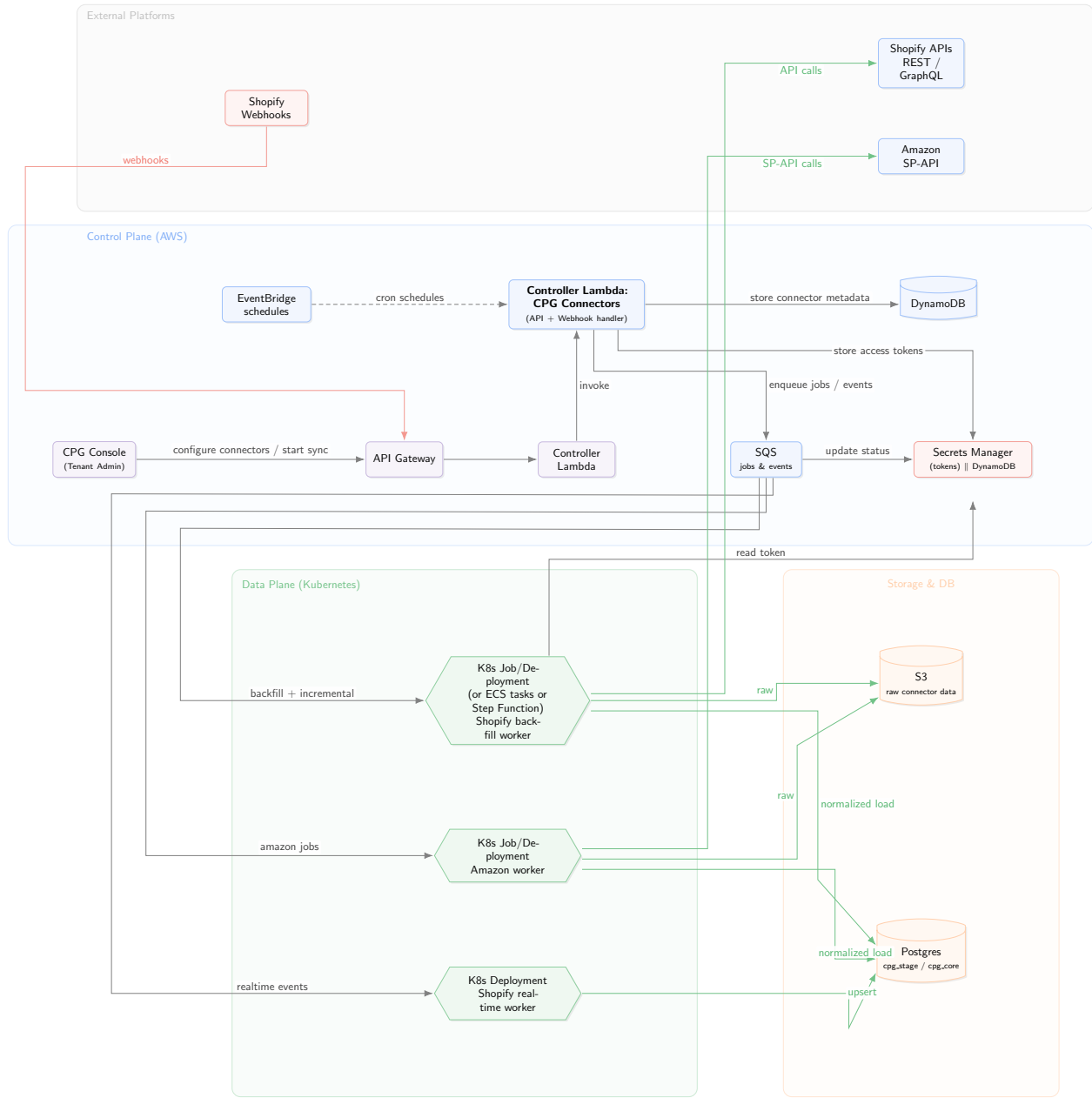


Figure 6. Connector ingestion architecture. The CPG Console drives connector configuration through the control plane (API Gateway → Controller Lambda → CPG Connectors), which stores metadata in DynamoDB, credentials in Secrets Manager, and enqueues job intents into SQS. The data plane executes on Kubernetes via three worker types: short-lived Jobs for Shopify backfills, Jobs/Deployments for Amazon syncs, and an always-running Deployment for Shopify real-time webhook consumption. All workers write raw data to S3 and normalized data to Postgres (`cpg_stage` / `cpg_core`). EventBridge provides cron scheduling for Amazon pulls and webhook routing for Shopify events.

Control plane responsibilities. When a tenant connects a source (e.g., Shopify), the request flows through API Gateway into the platform control plane and a domain controller service. That service:

- completes OAuth/auth, validates scopes, and handles webhook registration via EventBridge,

- stores connector metadata in DynamoDB (tenant, external identifiers, status, last sync timestamps),
- stores access tokens in Secrets Manager,
- and enqueues job intents into SQS (e.g., `job_type=historical_backfill` or `incremental_sync`) with `tenant_id` and connector identifiers [8].

The UI hits `/v1/cpg/connectors/...` endpoints which read/write job state from Dynamo so the tenant can see *sync in progress* / *completed* / *failed* per connector.

5.9.1. Data plane: historical backfill. For historical download, the data plane uses Shopify Bulk Operations to fetch large datasets efficiently. A `StartHistoricalPull` Lambda validates scopes, starts bulk operations, and stores `bulk_operation_id` and configuration in Dynamo. When Shopify completes the bulk operation, a webhook routes via EventBridge into a `ProcessBulkResult` Lambda [5] that:

- downloads the JSONL result,
- writes it to `S3/raw/` (partitioned by shop+date),
- and writes staging extracts into `cpg_stage` [6].

Normalization into the canonical model is executed by either Step Functions, ECS ETL, or a container worker on Kubernetes (depending on workload shape and cost profile) [3, 4, 11, 15].

5.9.2. Data plane: near-real-time (webhooks + always-running workers). For ongoing updates, we combine Shopify webhooks with always-running workers on Kubernetes:

- Shopify calls our webhook URLs (API Gateway → Lambda),
- the webhook handler validates the HMAC signature, maps shop domain → tenant/connector in Dynamo,
- and writes a compact event into SQS (optionally storing full payload in S3 for replay).

On Kubernetes, an always-running `shopify-realtime-worker` consumes events, hydrates details when needed, and applies upsert semantics into `cpg_stage` / `cpg_core`.

To cover missed webhooks, we can run periodic reconciliation (EventBridge cron or Kubernetes cron): scan active connectors, enqueue an `incremental_sync` job with a window since `last_incremental_sync_at`, and process it with the same worker image. This yields near-real-time freshness without depending on Airflow or long-lived Lambdas.

5.9.3. Amazon follows the same shape. Amazon SP-API does not provide the same webhook story, so everything is scheduled: EventBridge triggers a small coordinator Lambda that lists active Amazon connectors and enqueues backfill/incremental jobs in SQS per seller account. A Kubernetes worker image handles both backfills (short-lived Jobs) and incremental syncs (Jobs or a Deployment), writing raw to S3 and populating `cpg_stage` / `cpg_core`.

5.9.4. Role of Airflow. Airflow is intentionally *not* in the Shopify/Amazon datapull path. We use Airflow later for offline analytics and forecasting *on top of* `cpg_core` (nightly demand forecasts, promo elasticity jobs, heavy batch analysis), keeping ingestion API-driven and multi-tenant by design.

5.9.5. Tenant isolation in Postgres: tenant_id + optional RLS safety net. Unlike schema-per-tenant systems, the CPG module uses shared schemas (`cpg_stage`, `cpg_core`) with `tenant_id` BIGINT NOT NULL on tenant-owned tables. The schema DDL makes this explicit across dimensions and facts (e.g., `cpg_core.fact_order_line` includes `tenant_id`):

```

1 CREATE SCHEMA IF NOT EXISTS {schema_prefix}_stage;
2 CREATE SCHEMA IF NOT EXISTS {schema_prefix}_core;
3
4 CREATE TABLE IF NOT EXISTS {schema_prefix}_core.fact_order_line (
5     order_line_pk BIGSERIAL PRIMARY KEY,
6     tenant_id     BIGINT     NOT NULL,
7     order_pk      BIGINT     NOT NULL REFERENCES {schema_prefix}_core
8     .fact_order_header(order_pk),
9     sku_id        BIGINT     NOT NULL REFERENCES {schema_prefix}_core
10    .dim_sku(sku_id),
11     external_order_item_id VARCHAR(255),      -- Shopify line_item.id, Amazon
12     OrderItemId, etc.
13 );
14
15 CREATE INDEX IF NOT EXISTS fact_order_line_tenant_sku_idx
16 ON {schema_prefix}_core.fact_order_line (tenant_id, sku_id);

```

Listing 14. Shared schemas + tenant_id column.

At runtime, the safest pattern is belt-and-suspenders: queries explicitly filter by `tenant_id`, and row-level security (RLS) can be enabled as a backstop. A common approach is to bind tenant scope into the DB session using a custom setting (GUC):

```

1 -- application sets scope per request
2 SET LOCAL app.tenant_id = '<tenant_id>';
3
4 ALTER TABLE {schema_prefix}_core.fact_order_line ENABLE ROW LEVEL SECURITY;
5
6 CREATE POLICY tenant_isolation_fact_order_line
7 ON {schema_prefix}_core.fact_order_line
8 USING (tenant_id = current_setting('app.tenant_id')::BIGINT);

```

Listing 15. Optional RLS backstop: bind tenant scope into the session and enforce via policy.

This is the same paper thesis in SQL form: execution may be fast, but scope must be explicit.

5.10. Optima: heavy compute as container services, governed by serverless wrappers.

Some analytics do not fit inside Lambda without turning the deployment into a gym membership: NumPy/pandas/CVXPY-class workloads either exceed practical packaging limits or produce unacceptable cold starts. We therefore implement Optima-style modules as *container-native compute* governed by the same control plane contracts as the rest of the platform.

The pattern. Optima follows a three-part architecture that is intentionally reusable:

- **Thin serverless wrapper:** validates identity, derives effective context, enforces licensing/seat gates, and emits a job intent (scope + parameters).
- **Heavy container compute:** executes optimization/simulation/forecast workloads on EKS/ECS/Fargate with warm dependencies and predictable runtime behavior.
- **Artifact-first I/O:** large inputs/outputs move via S3 (often with presigned upload URLs), keeping API Gateway and the control plane slim and reliable [6,7].

Synchronous vs asynchronous UX.. For small computations, the wrapper can call the service synchronously. For heavy tasks, the wrapper returns immediately with a `job_id` and a status endpoint; results are stored as S3 artifacts and referenced by metadata. This avoids API Gateway timeout constraints while preserving a clean user experience.

Why. Optima is a proof that the control plane is not tied to a single execution substrate. The same governance primitives—module gating, effective context, scope envelope, auditability—apply whether execution happens in a Lambda query plane (Horizon/Grid), a Kubernetes service (DataMap), a connector worker fleet, or a heavy compute service (Optima). The platform is cohesive because governance is cohesive.

6. SECURITY AND DEFENSE-IN-DEPTH

A multi-tenant platform’s primary security objective is not “no bugs.” It is: *no tenant confusion*. Most catastrophic failures start as mundane mis-scoping.

We therefore design defenses around a concrete threat model.

6.1. Threat model (what we actually defend against). We focus on:

- **Forged tenancy context:** a client attempts to select a tenant/role they do not own.
- **Confused deputy:** a downstream service performs actions for the wrong tenant because it trusted caller-provided scope.
- **Surface drift:** clients discover and call endpoints that are not intended to exist for their effective context.
- **Accidental operator damage:** support/QA performs real actions while impersonating the wrong tenant.

6.2. Audit-Ready Foundations. This paper is not a compliance report, but production reality is constrained by compliance-grade expectations: clear boundaries, clear evidence, and recoverability that works under pressure. Those constraints are not paperwork; they are architecture.

Blast-radius discipline (accounts and regions). We operate with explicit separation boundaries (e.g., production vs. non-production isolation and regional separation for resilience). The key point is not the exact account topology; it is the design intent: reduce blast radius, simplify incident containment, and make boundaries legible to both operators and auditors.

Network segmentation as a default posture. Ingress is terminated at controlled edges (load balancers and WAF where applicable), while compute and data stores remain in private subnets without public IPs. When a platform is multi-tenant, “private by default” is not a preference; it is a survival trait.

Recoverability is engineered, not enabled. Backups exist at multiple layers (databases, object storage, volumes, and file systems), but the important detail is restore: we treat restore as a first-class operational capability and validate it regularly. In multi-tenant platforms, recoverability is part of tenant isolation: mistakes must be reversible.

Evidence is just observability with manners. Operational telemetry (logs, metrics, traces, configuration history) doubles as evidence when it is structured, retained, and attributable. That attribution is exactly why this paper emphasizes actor vs. effective context: it makes the system auditable by design, not by spreadsheet.

6.3. Layered controls: make tenant confusion expensive. Figure 7 summarizes the layered controls. Each layer is useful alone; together they form a system that is hard to misuse accidentally and hard to abuse intentionally.

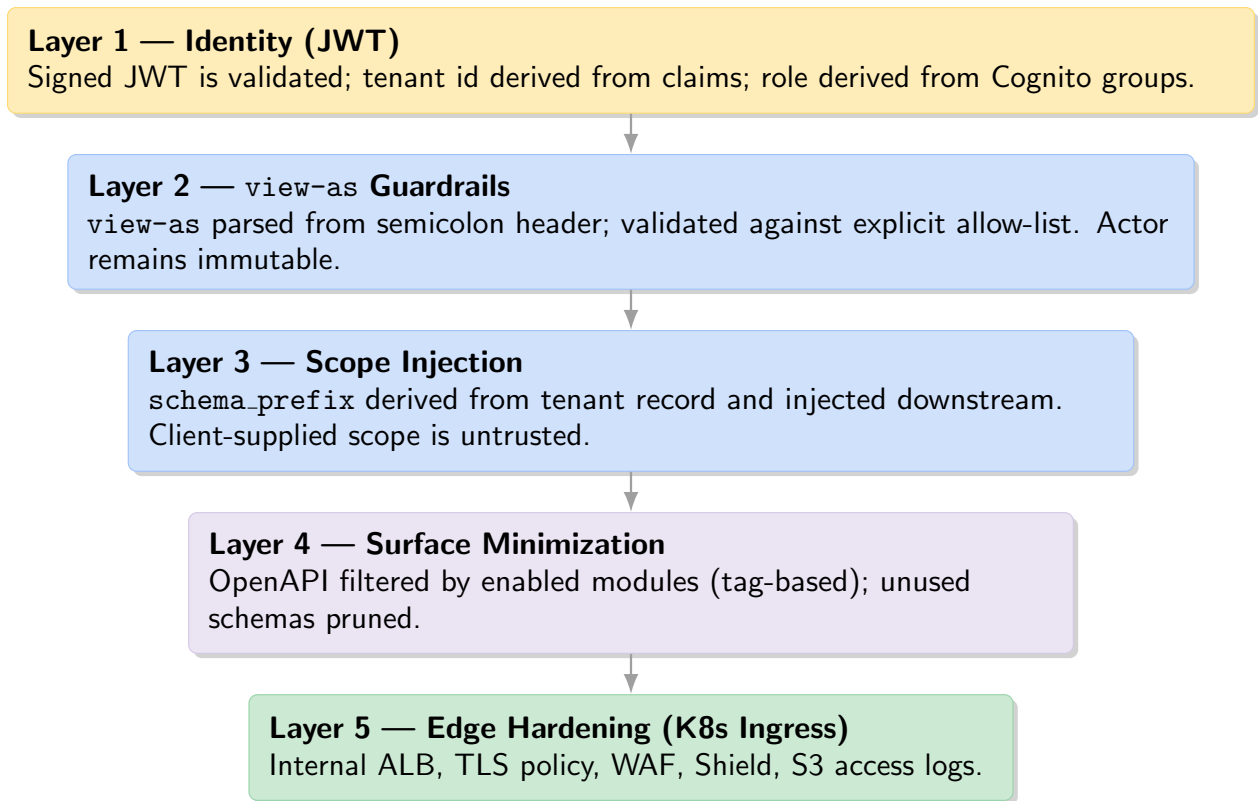


Figure 7. Defense-in-depth against tenant confusion: identity, view-as allow-lists, scope injection, minimized surface, and hardened edges. Narrowing layers indicate progression from identity verification (top) to network hardening (bottom).

6.4. Control plane hardening: explicitness over cleverness. The control plane adds baseline security headers on every response: `X-Frame-Options: DENY`, strict CSP (`default-src 'none'`), `X-Content-Type-Options: nosniff`, and HSTS. These are not glamorous, but they remove entire classes of browser-side surprises.

It also enforces that `view-as` must contain `tenant_id` and `role`, and denies requests that attempt to impersonate targets outside the allow-list. This turns impersonation from “header magic” into a controlled capability.

6.5. Scope injection: the anti-confused-deputy pattern. The most important security property of this platform is not that it denies access. It is that it *denies ambiguity*. When the control plane injects `schema_prefix` into downstream requests, it removes a class of vulnerabilities where the client “helpfully” tells the system what tenant it meant.

Downstream services can still validate scope, but they no longer need to guess it.

6.6. Kubernetes edge security: internal ingress + WAF + audit logs. Kubernetes ingress definitions for services like the Console and Airflow webserver use internal ALBs, enforce TLS policies, attach a regional WAF ACL, enable Shield, and stream access/connection logs to S3. This provides:

- reduced exposure (internal-only),
- consistent layer-7 protection (WAF),
- DDoS hardening (Shield),
- and an audit trail for forensic analysis (ALB logs).

6.7. Secrets and workload identity: IRSA/ESO for Kubernetes. Workloads obtain secrets via ESO and IRSA: service accounts are annotated with an IAM role ARN, enabling controlled access to Secrets Manager without embedding long-lived credentials in pods. This keeps secret access auditable and reduces credential sprawl.

6.8. Caveat: caching and multi-tenant correctness. ETags and short TTL caching make context contracts fast and cheap [17]. However, in multi-tenant systems, caching must never blur identity boundaries. We therefore treat these contracts as *safe for revalidation* and ensure the ETag is derived from the full response body (which includes effective context). In environments with shared intermediaries, additional cache controls (e.g., `private`) may be appropriate; the key invariant is: cached responses must not be reusable across principals.

6.9. Summary: security is mostly architecture. The security story here is not a single trick. It is a refusal to allow tenancy to be implicit. The control plane makes context explicit, the data plane executes within that context, and the edges are hardened so mistakes are contained.

Or, in operational terms: fewer "how did this happen?" incidents, and more "this couldn't have happened" guarantees.

7. OPERATIONS AND RELIABILITY

The control plane/data plane separation is not only a design preference; it is an operations strategy. When the platform misbehaves (and it will), we want failures to be legible: *what decided* and *what executed* should be distinguishable without reading tea leaves in CloudWatch at 03:00.

7.1. Operational invariants. We operate the system around invariants that make incidents less creative:

- **I1 (No implicit tenancy):** every downstream call has explicit tenant scope derived from authenticated context (e.g., `schema_prefix`).
- **I2 (Control plane decides):** capability and scope are determined in the control plane and forwarded as inputs. Compute services do not infer tenants.
- **I3 (Contracts are cache-friendly):** high-frequency contracts (notably `/v2/me`) support conditional requests via ETag/If-None-Match [17].
- **I4 (Actor remains attributable):** `view-as` never overwrites the actor; logs always contain both actor and effective context.

These invariants are boring in the best possible way: they make debugging mechanical.

7.2. Execution fabric: one platform, multiple shapes. A platform that runs only one kind of workload is either very small or lying. In CAUSIFY PLATFORM, different workload shapes map to different execution substrates:

- **Control plane Lambda:** high fan-in governance calls (auth/context), stable contracts, strict middleware enforcement [12, 13].
- **Module Lambdas (Horizon/Grid):** bursty interactive query planes with strict tenant scoping and fast scaling characteristics.
- **Kubernetes services (EKS):** long-running services, schedulers, and stateful components that benefit from steady orchestration [4].
- **Airflow + ECS/Fargate:** bursty bounded jobs where isolation is valuable and cluster debt is undesirable [3, 11].

Figure 8 presents these layers visually.

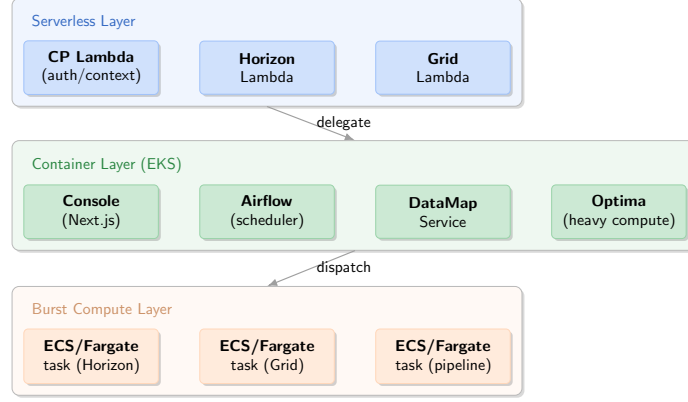
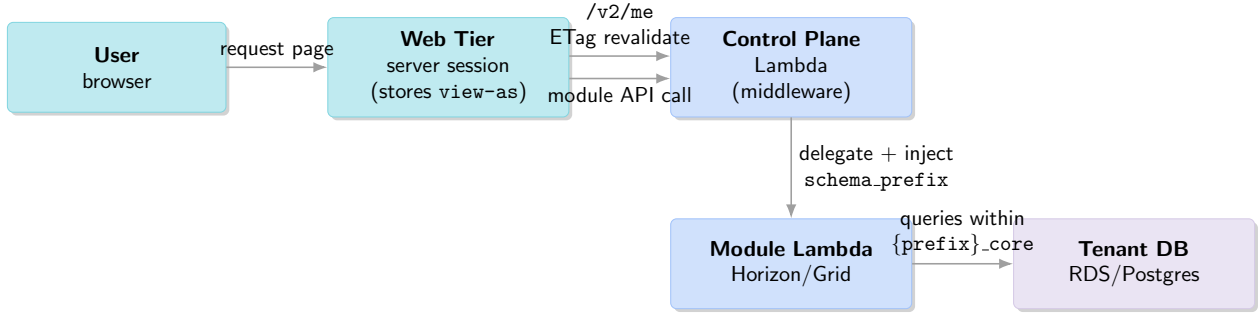


Figure 8. Workload topology: serverless functions handle governance and bursty queries; containers run long-lived services; Fargate provides isolated burst compute dispatched by Airflow.

Component	Purpose	Scale pattern	Tenant source	scoping	Operational failure mode
Control (Lambda)	Plane context, policy materialization, surface minimization	spiky, fan-in, cacheable	JWT + derived role; <code>view-as</code> allow-list; <code>schema_prefix</code>	group-derives	cold starts, upstream auth issues, config parsing failures
Horizon Lambda	interactive query plane (KPIs, etc.)	bursty dashboard traffic	<code>tenant_id</code> + derived DB scope (e.g., <code>schema_prefix</code>); re-fuses missing scope		DB connection pressure, slow queries, missing scope rejected early
Grid Lambda	interactive analytics / signals	bursty; periodic spikes	<code>tenant_id</code> + derived DB scope (e.g., <code>schema_prefix</code>); scope injected by control plane		slow queries; stale caches; scope mismatch surfaces as explicit error
DataMap Service (EKS)	studio workflow + APIs	steady + autoscaled	request context from platform + persistent storage model		resource contention, rollout issues, noisy-neighbor within cluster
Optima (EKS/ECS)	compute-heavy optimization / batch analytics	steady or bursty (async)	<code>tenant_id</code> + job intent (and optional artifact pointers in S3)		queue backlogs, capacity/provisioning tails, long-running job timeouts
Airflow + ECS/-Fargate	pipelines + bounded jobs	scheduled bursty	+ job params include explicit tenant scope (passed as env/args)		capacity/provisioning failures; retries; long tail run times

Table 4. Module backend comparison: different workload shapes, one scoping principle.

7.3. Delegation pipeline: scope injection as an operational safety rail. The most useful operational feature is not fancy dashboards; it is the elimination of ambiguous scope. Figure 9 shows the delegation pipeline for a module request where the control plane forwards to a module Lambda and injects derived tenant scope.



Key safety rail: the client does not choose tenant scope. The control plane derives it from identity/view-as, injects it downstream, and logs both actor and effective context.

Figure 9. Control plane delegation with scope injection. Tenant scope is derived, not supplied.

7.4. Observability: attribution beats volume. A multi-tenant platform does not need more logs; it needs more *attribution*. We treat actor and effective context as first-class log fields and propagate request IDs end-to-end (web tier → control plane → module backends). This keeps incident timelines coherent and prevents the classic failure mode where operators spend hours proving what tenant a request belonged to.

7.5. Change management: policy edits are production changes. Because tenant/module configuration is loaded at runtime, changes to it are treated as production changes. Operationally, that means:

- schema validation and strict JSON parsing (fail fast, do not half-render),
- reviewed changes (diffable, rollbackable),
- and staged rollouts when altering module surfaces.

The platform remains evolvable because governance changes are data operations, but they remain safe because those data operations are handled with the same discipline as code.

8. EVALUATION

This section evaluates whether the control-plane/data-plane boundary achieves its goals: explicit tenancy, minimized surface drift, operational clarity, and acceptable overhead. We report evaluation as a set of measurable questions with concrete metrics where available.

8.1. Q1: Is the control plane fast enough for interactive use? The web tier calls `/v2/me` frequently. The control plane should keep that contract cheap; here we report `/v2/me` latency from a load of 100 total requests with concurrency 10, as summary statistics (Table 5) and percentiles (Table 6).

Metric	Value (ms)
Mean	1415.15
Std Dev	372.52
Min	1001.21
Max	2594.79

Table 5. `/v2/me` latency: summary statistics (100 requests, concurrency 10).

Percentile	Value (ms)
p1	1008.74
p5	1025.96
p10	1042.47
p25	1077.33
p50	1498.69
p75	1541.80
p90	1842.08
p95	2341.29
p99	2510.98

Table 6. /v2/me latency: percentiles (same run as Table 5).

8.2. Q2: Does scope injection prevent tenant confusion? We treat “tenant confusion” as an incident class: any request where a component touches data outside the effective tenant scope.

We validate scope injection in two ways:

- **Static checks:** module routers require `schema_prefix` and refuse to operate without it.
- **Runtime correlation:** logs include actor tenant, effective tenant, and `schema_prefix`; anomalies are detectable via simple queries (e.g., `schema_prefix` not matching effective tenant).

Metric. Count occurrences of mismatched scope fields across the request path. The target is zero; the practical goal is “approaches zero and spikes are explainable.”

Code evidence. The injection chain is traceable in the codebase. `auth_middleware` (`middleware.py`) derives `schema_prefix` from the tenant record, not from any client-supplied header or query parameter. The controller injects this value into every downstream Lambda invocation: `services/horizon.py` and `services/grid.py` both pass it as a required parameter in the internal request payload. Horizon Lambda refuses to operate without `schema_prefix`, raising a `BadRequestError` if the value is missing. This means a confused-deputy scenario requires *two* independent failures: the controller must mis-derive the prefix *and* the module Lambda must accept it.

8.3. Q3: Does OpenAPI filtering reduce surface drift? Surface drift occurs when clients can discover or call endpoints for modules that are disabled. We measure:

- number of module endpoints present in the raw OpenAPI vs filtered OpenAPI,
- attempted calls to disabled module endpoints (should be denied and ideally rare),
- UI-to-spec alignment: endpoints used by the client must exist in the filtered spec.

Result shape. In a healthy system, filtered OpenAPI removes a large fraction of endpoints for tenants with limited modules, and attempted calls to disabled endpoints remain near zero (ideally because clients never see them).

Code evidence. Two functions in `services/openapi.py` implement the filtering. `filter_openapi_paths()` removes every path whose tags do not intersect the tenant’s enabled module set. `filter_openapi_schemas()` then recursively prunes schema definitions that are no longer referenced by any surviving path, preventing information disclosure about features the tenant has not licensed. Critically, the filtering uses the *effective* context (the `view_as` identity after impersonation resolution), not a static configuration file. This means the same deployment serves different API surfaces to different tenants without a redeploy, and an impersonating support agent sees exactly the surface available to the tenant they are assisting.

8.4. **Q4: Is the control plane actually “thin”?** The architecture claims that governance is concentrated into a thin layer. We validate this by measuring the control plane’s code footprint and comparing it to the data plane infrastructure it governs.

Component	Size
Total control plane Python files	37 files
Total control plane lines of code	~6,174
Middleware stack (<code>core/middleware.py</code>)	248 lines
Auth modules (JWT + decorators)	133 lines
All six routers combined	2,420 lines
Pydantic data models	75 model classes
<i>Data plane for comparison:</i>	
Kubernetes YAML manifests	743 files
Kubernetes namespaces / applications	12
Terraform infrastructure modules	18

Table 7. Control plane code footprint vs. data plane infrastructure. The entire governance layer (authentication, impersonation, scope injection, module gating, surface filtering, and response hardening) fits in ~6,200 lines of Python, governing a data plane defined by 743 manifests across 12 applications.

The eight distinct defense layers (identity, impersonation, scope injection, module gating, surface filtering, response hardening, transport, and network) are implemented in approximately 300 lines of middleware code. “Thin” is not a metaphor; it is measurable.

8.5. **Q5: Does the boundary reduce operational churn?** We evaluate whether governance moves from redeploys to data operations:

- number of deployments triggered by tenant/module enablement changes,
- mean time from “policy edited” to “platform reflects change,”
- incidence of production hotfixes caused by policy drift across services.

Expected outcome. When the control plane owns tenancy and capability contracts, tenant onboarding and module toggles should require fewer code changes and fewer redeploy cycles.

Table 8 shows the six most common tenant lifecycle operations and whether each requires a code deployment or only a data change.

Operation	Requires Deploy?	Mechanism
Add new tenant	No	DynamoDB record insert
Enable/disable module	No	DynamoDB record update (module list)
Change module layout	No	DynamoDB <code>moduleLayout</code> JSON field
Add impersonation target	No	Cognito group assignment
Change schema prefix	No	DynamoDB tenant record update
Add a <i>new module type</i>	Yes	New router + <code>@requires_module</code> tag

Table 8. Tenant lifecycle operations. Five of six common operations are data-only changes that take effect without a code deployment, reducing operational churn.

8.6. **Q6: How much does surface minimization reduce the visible API?** The paper claims OpenAPI filtering reduces surface drift. We quantify the reduction by counting endpoints visible to tenants with different module configurations.

Tenant profile	Visible endpoints	Reduction
Full platform (all modules)	48	—
Single module (e.g., Grid only)	~16	~67%
Core only (no optional modules)	5	~90%

Table 9. API surface reduction by OpenAPI filtering. A tenant with a single enabled module sees roughly one-third of the total surface; a tenant with no optional modules sees only the five core endpoints (`/v2/me`, `/v1/me`, `/v1/tenant`, `/v1/openapi.json`, `walkthrough`).

Why this matters. In multi-tenant platforms, surface drift is not merely a documentation nuisance; every exposed endpoint is an attack surface and a support liability. A 67 to 90% reduction in visible endpoints for limited-module tenants eliminates entire classes of accidental discovery and confused-client bugs.

8.7. **Q7: What does governance concentration save?** To quantify the value of the control plane boundary, we consider a counterfactual: what if each data plane application independently implemented its own governance?

Concern	With CP	Without (per-svc)	Risk avoided
JWT validation	1 impl. (73 LoC)	12 impls.	Inconsistent token handling
Tenant lookup	1 DynamoDB call	12 DB lookups	Stale or missing tenant records
Impersonation logic	1 allow-list (70 LoC)	12 scattered checks	Partial or missing allow-lists
Module gating	1 decorator	ad-hoc per svc	Ungated endpoints
Schema prefix	1 injection point	12 trust-the-client	Confused-deputy risk $\times 12$
OpenAPI filtering	1 dynamic filter	impossible / manual	Permanent surface drift

Table 10. Counterfactual analysis: governance with vs. without the control plane boundary. The control plane replaces an estimated ~3,600 lines of scattered governance code (300 lines $\times$ 12 services) with a single 248-line middleware file.

The savings are not merely in lines of code; they are in *consistency*. Twelve independent implementations of tenant scoping means twelve opportunities for one of them to trust a client-provided `schema` parameter. One implementation means one place to audit, one place to patch, and one place to get right.

8.8. **Q5: Is governance centralized or scattered?** Section 2 identifies *policy sprawl*, the tendency for each service to re-derive tenancy and authorization independently, as a root cause of governance drift. This question evaluates whether the architecture achieves centralization in practice, not merely in diagrams.

Code evidence. We count the files in the codebase that make governance decisions, authentication, tenant resolution, module authorization, and scope derivation:

- **Authentication and context assembly:** `auth_middleware` in `middleware.py` (1 file). This single middleware resolves the actor identity, validates impersonation, and assembles the full effective context.

- **Module-level authorization:** the `@requires_module` decorator in `auth/decorators.py` (1 file). Every module-specific router endpoint (Horizon, Grid, DataMap) is gated by this decorator, which checks whether the effective context includes the required module; submodule-only endpoints also use `@requires_submodule`.
- **Scope derivation:** `schema_prefix` computation in `middleware.py` (same file as authentication). The prefix is derived from the tenant record and injected into downstream calls.

In total, **three governance mechanisms reside in two files**, both inside the controller Lambda. Module Lambdas (`horizon_lambda/`, `grid_lambda/`) contain *zero authentication or tenant-resolution middleware*. They receive a pre-validated scope and execute within it. This is the structural counterpart of Figure 2: governance is not merely “recommended to be centralized”, it is architecturally impossible for a module Lambda to improvise its own tenancy, because the relevant code does not exist there.

8.9. Q6: Does context separation support auditable impersonation? Design goal G1 requires that actor identity and effective context remain structurally separate, especially during impersonation. This question evaluates whether the separation is enforced and observable.

Code evidence. The `/v2/me` endpoint (`routers/user.py`) returns a response containing distinct `actor` and `view_as` objects. These two structures are never merged: the actor reflects the authenticated user; `view_as` reflects the tenant context they are operating in. When no impersonation is active, both point to the same identity, but the structural separation is still present, consuming code never needs to branch on “am I impersonating?” to find the right field.

Before accepting an impersonation request, `auth_middleware` validates the target tenant against the user’s `available_impersonations` list. If the target is not in the list, the request is rejected before any downstream processing occurs.

Impersonation is also visible in telemetry. APM labels include `is_impersonating`, `actual_tenant`, and `viewing_tenant` (`user-context.ts`), ensuring that support sessions are distinguishable from normal traffic in traces and dashboards. Frontend tests (`profile.spec.ts`) validate the `actor/view_as` structure, confirming the contract is enforced across the stack.

8.10. Interpretation. The evaluation is not a benchmark contest; it is a correctness and operability check. Together, Q1–Q6 cover all five design goals:

- **G1** (explicit context) is evaluated by Q6: actor and effective identities are structurally separate, impersonation is validated and visible in telemetry.
- **G2** (server-injected scoping) is evaluated by Q2: `schema_prefix` is derived server-side and injected into every downstream call, with Horizon enforcing its presence.
- **G3** (minimized surface) is evaluated by Q3: OpenAPI filtering removes endpoints and schemas per tenant context.
- **G4** (cache-friendly contracts) is evaluated by Q1: ETag-based revalidation reduces payload transfer for `/v2/me`.
- **G5** (operational clarity) is evaluated by Q4 and Q5: governance changes are data operations (Q4), and policy logic is centralized in two files within the controller Lambda (Q5).

Q5 additionally confirms that governance centralization is structural, not merely conventional: module Lambdas contain zero authentication or tenant-resolution code. Q6 confirms that context separation is maintained across the full stack, from middleware to APM labels to frontend tests. If `/v2/me` is fast, scope mismatches are absent, surface drift is reduced, governance is centralized, and impersonation is auditable, then the boundary is doing its job: keeping governance explicit and keeping compute focused on execution.

The quantitative results reinforce this: the entire governance layer fits in ~6,200 lines of Python (Q4), five of six common tenant operations require no deployment (Q5), surface filtering removes 67 to 90% of endpoints for limited-module tenants (Q6), and the alternative, per-service governance, would scatter ~3,600 lines of policy logic across twelve independent implementations (Q7). The boundary is not merely conceptually clean; it is measurably economical.

9. RELATED WORK

Control plane vs. data plane as a systems pattern. The separation of decision-making from execution is foundational in systems architecture. Kubernetes formalizes this distinction inside the cluster: control plane components manage desired state while worker nodes execute workloads [19, 20]. Managed Kubernetes offerings extend the idea by externalizing control plane operations behind a service boundary [4]. This paper applies the split to multi-tenant governance: tenancy and scope are decisions; compute is execution.

Multi-tenant Kubernetes frameworks. Prior work has explored explicit control-plane/data-plane isolation for multi-tenant Kubernetes, including frameworks that provide per-tenant control planes while sharing underlying compute resources (e.g., VirtualCluster) [23]. Our focus differs: we do not attempt to re-slice Kubernetes itself; we keep tenant governance in a platform control plane and propagate explicit scope into execution substrates (Kubernetes, ECS/Fargate, and module backends).

Policy engines and authorization languages. OPA provides a general-purpose policy engine that centralizes decision logic and evaluation [21]. Cedar (and Amazon Verified Permissions) formalize policy evaluation with a dedicated language and service boundary [9, 16]. Our work is compatible with these systems but focuses on a narrower objective: producing *cacheable governance contracts* (context, surface, scope) that downstream systems consume without improvising tenancy.

Authorization at scale. Zanzibar demonstrates authorization as a global consistency problem and highlights the importance of uniform interpretation of identity and context across services [22]. While CAUSIFY PLATFORM does not implement Zanzibar’s relationship store, the lesson applies directly: once systems scale, “context correctness” becomes a distributed property, not a local detail. Our design addresses this through explicit contracts and scope injection rather than relationship evaluation.

Serverless APIs for governance. Serverless functions are a natural fit for governance endpoints: spiky traffic, small payloads, and a strong need for structured logs [12, 13]. The main tradeoff is cold-start tail latency; we mitigate this by keeping the control plane thin and using HTTP conditional requests for frequent identity contracts [17].

Workflow orchestration and batch execution. Airflow is a widely used workflow platform for building and operating batch-oriented pipelines [10]. Executing tasks in isolated containers (e.g., ECS/Fargate) is a pragmatic strategy for burst compute where operational simplicity and isolation matter [3, 11]. In our architecture, workflows execute jobs, but job scope is derived from authenticated tenant context rather than client-supplied parameters.

10. LIMITATIONS AND FUTURE WORK

The control-plane/data-plane split buys clarity, but it does not buy immortality. It changes where complexity lives: less in scattered services, more in contracts, schemas, and governance tooling. This section names the sharp edges and where we intend to sand them down.

10.1. Contract evolution and versioning. Today, `/v2/me` is a stable contract that the web tier depends on heavily. As the platform grows, we must avoid two failure modes:

- **Contract drift:** clients assume fields that are no longer correct.
- **Contract explosion:** every new feature invents a new “me-like” endpoint.

Future work includes explicit contract versioning (beyond path versioning), schema documentation, and compatibility tests that run in CI against representative tenant/role contexts.

10.2. Cache correctness: cheap revalidation without accidental sharing. Conditional requests (ETag/If-None-Match) make /v2/me cheaper [17]. However, multi-tenant systems must treat caching as a correctness topic, not a performance topic. Future work includes:

- tightening cache directives where shared intermediaries exist,
- adding a policy version field so caches can be invalidated deterministically,
- and building explicit "cache safety tests" that ensure responses are not reusable across principals.

10.3. Stronger scope propagation: beyond schema_prefix. Today, `schema_prefix` is an effective scoping signal because it is derived from authenticated context and injected downstream. The limitation is that it is *conventional*: downstream code must continue to respect it. Future work includes stronger propagation mechanisms:

- cryptographically signed scope tokens validated by module backends,
- and/or a standardized "tenant context envelope" passed through every hop (web tier → control plane → module → database).

The goal is not to add ceremony; it is to make tenant confusion mechanically difficult.

10.4. Policy authoring: humans deserve better than raw JSON. When policy is data, authoring experience becomes a production dependency. Strict parsing is good, but future work should include:

- versioned JSON schemas and offline validation tools,
- a review workflow with diffs and previews for tenant/module changes,
- and a "staged rollout" mechanism so policy changes can be deployed gradually.

This keeps governance flexible without making it fragile.

10.5. OpenAPI filtering: keeping surface minimization provable. Filtering OpenAPI reduces surface drift, but it introduces its own correctness obligation: the filtered spec must match runtime behavior. Future work includes:

- automated spec-vs-runtime consistency checks,
- a test harness that enumerates enabled modules and asserts that endpoints are both (i) present in the filtered spec and (ii) callable when authorized,
- and explicit "module tag" governance so tag mistakes cannot leak endpoints.

10.6. Airflow & batch execution: toward a first-class job contract. Airflow and ECS/Fargate provide a pragmatic batch substrate, but jobs currently encode scope as parameters (env/args). Future work includes a first-class "job intent" contract emitted by the control plane:

- a structured payload that includes tenant scope, job identity, and trace context,
- consistent retry semantics and failure classification,
- and standardized audit logs that tie job execution back to actor/effective context.

This would make batch execution a citizen of the governance model, not a tolerated exception.

10.7. Multi-region control plane strategy. The platform already runs in multiple regions for resilience (as operational reality demands). Future work includes making the control plane explicitly region-aware:

- routing strategy for the web tier,
- failure-mode playbooks (what happens when a region is unhealthy),
- and tested procedures for policy/config replication where necessary.

In summary: the core pattern holds, but the tooling around it must mature. A thin control plane stays thin only if we refuse to turn it into a dumping ground.

KEY TAKEAWAYS

- **Make tenancy explicit everywhere.** Treat tenant context as a first-class artifact: keep *actor* identity immutable, represent *effective* context explicitly, and refuse to let downstream systems guess.
- **Decide at the edge, execute in the fabric.** A thin serverless control plane is a governance engine: it authenticates, derives scope, materializes contracts, and minimizes surface. The container data plane should do what it does best: run workloads.
- **Scope injection beats scope suggestion.** Derive scoping signals (e.g., `schema_prefix`) from authenticated context and inject them into delegated calls and jobs. This is the practical antidote to confused-deputy failures.
- **Stop advertising what you will deny.** Filter OpenAPI and gate routers by enabled modules so clients do not discover endpoints that should not exist for their context.
- **Make "who am I?" cheap enough to ask often.** Use ETags and conditional requests for high-frequency context contracts. In multi-tenant systems, correctness plus caching is not an optimization; it is stability.

11. CONCLUSION

In multi-tenant platforms, the expensive bugs are rarely clever. They are mundane: a missing scope field, a stale spec, an endpoint that exists when it should not, a job that runs under the wrong context. The platform fails not because compute is hard, but because governance becomes ambiguous.

This paper described a production pattern that makes ambiguity unwelcome: a *serverless control plane* that materializes identity, effective context, capability surface, and tenant scoping into explicit contracts, paired with a *container data plane* that executes workloads without owning policy.

The results are practical:

- tenancy becomes explicit (actor vs. effective context),
- scope is injected rather than guessed (`schema_prefix` and friends),
- surface drift is reduced (filtered OpenAPI + module gating),
- and "who am I?" becomes cheap enough to ask often (ETag + conditional requests).

The thesis is simple:

Governance is a product feature. Put it in one place, make it explicit, and keep it boring.

The data plane can remain ambitious. The control plane should remain calm. In a world where platforms grow faster than documentation, calm is a competitive advantage.

REFERENCES

- [1] Amazon API Gateway, *Amazon api gateway documentation*. Accessed 2026-02-22.
- [2] Amazon DynamoDB, *Amazon dynamodb documentation*. Accessed 2026-02-22.
- [3] Amazon Elastic Container Service (ECS) API Reference, *Runtask — amazon elastic container service api reference*. Accessed 2026-02-22.
- [4] Amazon Elastic Kubernetes Service (EKS), *Amazon elastic kubernetes service (eks) documentation*. Accessed 2026-02-22.
- [5] Amazon EventBridge, *Amazon eventbridge documentation*. Accessed 2026-03-03.
- [6] Amazon S3, *Amazon s3 documentation*. Accessed 2026-03-03.
- [7] Amazon S3 presigned URLs, *Generating presigned urls for amazon s3*. Accessed 2026-03-03.
- [8] Amazon Simple Queue Service (SQS), *Amazon simple queue service (sqs) documentation*. Accessed 2026-03-03.

- [9] Amazon Verified Permissions, *Amazon verified permissions documentation*. Accessed 2026-02-22.
- [10] Apache Airflow Project, *Apache airflow documentation*. Accessed 2026-02-23.
- [11] AWS Fargate, *Architect for aws fargate for amazon ecs*. Accessed 2026-02-22.
- [12] AWS Lambda, *Aws lambda documentation*. Accessed 2026-02-22.
- [13] AWS Lambda Powertools, *Powertools for aws lambda (python)*. Accessed 2026-02-22.
- [14] AWS Secrets Manager, *Aws secrets manager documentation*. Accessed 2026-02-22.
- [15] AWS Step Functions, *Aws step functions documentation*. Accessed 2026-03-03.
- [16] Cedar Policy Language Authors, *Cedar policy language reference guide*. Accessed 2026-02-22.
- [17] Roy T. Fielding, Mark Nottingham, and Julian Reschke, *RFC 9110: HTTP semantics*, 2022. Accessed 2026-02-22.
- [18] Norman Hardy, *The confused deputy (or why capabilities might have been invented)*, Acm sigops operating systems review, 1988. Classic discussion of the confused deputy problem.
- [19] Kubernetes Cluster Architecture, *Cluster architecture*. Accessed 2026-02-23.
- [20] Kubernetes Documentation, *Kubernetes components*. Accessed 2026-02-23.
- [21] Open Policy Agent Contributors, *Open policy agent (opa) documentation*. Accessed 2026-02-22.
- [22] Ruoming Pang, Manuel Cáceres, et al., *Zanzibar: Google’s consistent, global authorization system*, Proceedings of the usenix annual technical conference (atc), 2019. Accessed 2026-02-22.
- [23] Chao Zheng, Qinghui Zhuang, and Fei Guo, *A multi-tenant framework for cloud container services*, 2021. Accessed 2026-03-03.